960121

The Shopping Cart

Session 04

Objective

- In Session 2, we gave our app a **Memory** (Data).
- In Session 3, we gave it **Reflexes** (Events).
- For **Session 4: The Shopping Cart (State & Persistence)**, we reach a critical architectural milestone. In previous sessions, our data was "volatile"—if the user refreshed the page, their filters and choices vanished.
- Now, we will learn how to make an application "remember" using **State Management** and **Browser Storage**.

Maintaining Continuity

- In a full-stack context, **Maintaining Continuity** is the art of making a **stateless environment** (the web) feel like a continuous, personalized experience.
- For undergrads, this is where "coding" becomes "systems design." You aren't just storing a number; you are managing a **lifecycle**.

1. The Serialization Bridge

Data exists in different "states of matter."

- **In-Memory (Live)**: While the app is running, the cart is a collection of **JavaScript Objects**. It is fast, but volatile (gone on refresh).
- **On-Disk (Stored)**: LocalStorage only stores **Strings**.
- **The Logic**: You must master the "**Serialization Bridge**."
  - **Saving**: `JSON.stringify(cartArray)` turns the "live" objects into a string.
  - **Loading**: `JSON.parse(storedString)` turns that string back into "live" objects that the code can manipulate.

2. The Hydration Lifecycle

In architectural terms, "**Hydration**" is the process of filling your UI with preserved data when the application starts. You should know how to design a **Startup Sequence**:

- **Check**: Does the key `shopping_cart` exist in LocalStorage?
- **Validate**: Is the data there actually a valid array? (Critical thinking: what if the data is corrupted?)
- **Sync**: Set the global `cartState` variable to this saved data.
- **Render**: Immediately call the UI update functions so the user sees their items as soon as the page loads.

3. Distributed UI Synchronization

Continuity also means **Consistency**. If a user clicks "Add to Cart" on a product card, three different UI components must react to maintain continuity:

- The **Cart Sidebar** must show the new item.
- The **Navbar Icon** must increment the count
- The **Checkout Total** must recalculate.

The Architectural Insight: You don't write three different "update" functions. You write **one** "State Observer" pattern (even a simple version) that updates `cartState` and triggers a "refresh" across all components.

4. Continuity vs. Persistence

- **Continuity** (Session 4): Making sure the user doesn't lose progress during a single session or across refreshes (Local Storage).
- **Persistence** (Future Sessions): Making sure the user can access their cart from a different device (Database/Cloud).

localStorage

The `localStorage` property in JavaScript is part of the **Web Storage API** and allows you to store key-value pairs in a web browser. Unlike cookies, the data has no expiration date and persists even after the browser is closed or the device is rebooted.

localStorage

- **Capacity**: Most modern browsers allow approximately **5MB** to **10MB** of data per origin
- **String Only**: Stores data only in string format. To store objects or arrays, you must use `JSON.stringify()` for storage and `JSON.parse()` for retrieval.
- **Same-Origin Policy**: Data is isolated by protocol, domain, and port. For example, http://site.com cannot access data from https://site.com.
- **Synchronous**: Operations are synchronous and can block the main thread if handling very large datasets.

localStorage Core Methods

- `setItem(key, value)`: Adds a key and its corresponding value to the storage.

```
// Returns 'dark'
localStorage.setItem('theme', 'dark');
```

- `getItem(key)`: Retrieves the value associated with a specific key.

```
const theme = localStorage.getItem('theme');
```

localStorage Core Methods

- `removeItem(key)`: Deletes a specific item from the storage by its key.

```
localStorage.removeItem('theme');
```

- `clear()`: Completely empties all data stored for that origin (domain).

```
localStorage.clear();
```

How to Handle Non-String Data

`localStorage` only stores strings, but you can use JSON methods to work with complex data structures:

```
// Storing an object
const userPreferences = {
    theme: 'dark',
    fontSize: 14,
    notifications: true
};
localStorage.setItem('preferences', JSON.stringify(userPreferences));
// Retrieving and parsing the object
const storedPreferences = JSON.parse(localStorage.getItem('preferences'));
console.log(storedPreferences.theme); // Outputs: dark
```

localStorage vs sessionStorage

`localStorage` and `sessionStorage` are both parts of the **Web Storage API** used to store key-value pairs in the browser. While they share the same methods, they differ significantly in **lifespan** and **scope**.

- **Lifespan**: `localStorage` persists indefinitely, whereas `sessionStorage` lasts only until the tab/window is closed.
- **Scope**: `localStorage` data is shared across all tabs/windows of the same origin, while `sessionStorage` is strictly limited to the creating tab.

localStorage vs sessionStorage vs Cookies

Feature	localStorage	sessionStorage	Cookies
Persistence	Until manually deleted	Until the tab is closed	Acooding to Configuration
Scope	Shared across all **tabs** of the **same origin**	Limited to the specific **tab/session**	Controlled by the **Domain** and **Path** attributes
Capacity	~5-10MB	~5MB	~4KB
Data Types	String	String	String
API Complexity	Simple	Simple	Moderate

Security Warning

- According to **security best practices**, you should never store **sensitive information (like passwords or authentication tokens)** in `localStorage`.
- It is vulnerable to Cross-Site Scripting (XSS) attacks because any JavaScript running on the page has full access to the storage.

Referance

- **Window: localStorage property** (https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage)
- **How to Use localStorage in JavaScript to Save and Retrieve Data** (https://strapi.io/blog/how-to-use-localstorage-in-javascript)

Class Activity: The "Add to Cart" Logic

1. **The Scenario:**
   - **The Challenge**: A user adds "Product A" to the cart. Then they add "Product A" again.
   - **The Business Logic Dilemma**: Do we show "Product A" twice in the list, or do we update a `quantity` property? Most professional e-commerce sites update the quantity.

The "Add to Cart" Logic

2. (15 min.) UML:

- Draw an **Activity Diagram** for `addToCart(productID)` function.
- **The Decision Diamond**: you must include a conditional:
  - Does this ID already exist in the cart array?
  - If **YES** → Increment quantity
  - If **No** → Push new object to the array with quantity = 1

The "Add to Cart" Logic

3. (10 min.) Peer Evaluation:

- Students swap drawings.
- **Critical Thinking Question**:
  - If your friend deletes an item from the cart, does their logic also update the total price?
  - Does it save to Local Storage immediately?"

The "Instant Response" Catalog

4. (5 min.) GenAI Prompt Engineering :

Prompt: (prepare for “Add to Cart” action)

*“I am using an **Event-Driven** approach. I have a parent div with ID `#catalog`. I want to use **Event Delegation** to listen for clicks on any button with the class `.add-to-cart`. When clicked, grab the `data-id` attribute (product ID) and update my `cart` object. Show me the JavaScript for the listener and the handler function with comments explaining how the code works.”*

The "Add to Cart" Logic

4. (5 min.) GenAI Prompt Engineering:

- Students use UML logic to craft a structured prompt.

The "Add to Cart" Logic

5. (20 min.) Integration:

- Open your project in VS Code.
- Create a new `cart.js` file or section.
- Link the "Add to Cart" buttons (from Session 3) to this new logic.

The "Add to Cart" Logic

6. (10 min.) Debugging & Testing:

- **The "Refresh Test"**: add items, refresh the page, and realize the cart is empty.
- **The "Incognito Test"**:
  - Add items into cart,
  - Open your site in an **Incognito/Private window**.

- **Critical Thinking Question:** "Why did our continuity break? What is missing?"

The "Add to Cart" Logic

You must now write the "Load" logic: On page load, check Local Storage; if data exists, parse it back into the cart state.

6. (10 min.) GenAI Prompt Engineering:

The "Add to Cart" Logic

7. (5 min.) Version Control:

- Terminal Command:
  - `git add`
  - `git commit -m "feat: implement persistent shopping cart with localstorage.”`
  - `git push.`
- VS Code:
- GitHub Desktop:

Key Takeaway

- **State as the Single Source of Truth:** The UI is just a "slave" to the cart array. If the array changes, the UI follows.
- **Data Integrity:** When we move data to Local Storage, we are "Serializing" it. You must know that computers see "Objects" and "Strings" very differently.
- **User Expectations:** Persistence is a business requirement. A cart that empties itself on a refresh is a bug that loses money.